25dce049 department of computer engineering

Practical – 7

PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

Problem Definition

Process the submitted StudentHub registration/contact form using PHP. Receive the data with the POST method, sanitise and validate every input on the server, store each valid record in a CSV file and a JSON file with safe file writing, and display clear success and error messages. Use registration/contact form inputs as test data.

A. Theory

1. Is the form submitted using POST?

Answer: Yes. The form uses method="post" and action="register.php". POST sends the data in the request body, so it does not appear in the URL, browser history, or server logs, and it is the correct method for forms that create or change data. The script confirms $_SERVER['REQUEST_METHOD'] === 'POST' before processing; other methods except GET receive HTTP status 405.

2. Are inputs validated and sanitised server side?

Answer: Yes. collect_input() reads every value from $_POST, accepts only strings (arrays are ignored), trims it, removes HTML tags with strip_tags(), removes control characters, and collapses extra white space. validate_registration() then checks the name, email, and mobile number with regular expressions and filter_var(), checks course, year, and gender against whitelists with in_array(..., true), and limits the message length. Output is escaped with htmlspecialchars() when it is displayed.

3. Is file writing handled safely?

Answer: Yes. File names are constants and never come from user input. The data folder is created if missing and is protected by .htaccess. Files are opened with fopen(), locked with flock(LOCK_EX) while writing so that two users cannot corrupt the file, written with fputcsv() or json_encode(), and closed. Failures are detected (fopen, flock, and fwrite results are checked), logged with error_log(), and reported to the user with a general message. A corrupt JSON file is never overwritten, and CSV cells that begin with =, +, -, or @ are prefixed to prevent spreadsheet formula injection.

4. Are success and error responses displayed clearly?

Answer: Yes. Errors are shown beside each field and in an error summary with links; the entered values are kept so the user does not retype everything; and suitable HTTP status codes are sent (422 for validation errors, 403 for a bad token, 500 for a storage failure). After success the script redirects (Post/Redirect/Get) and shows a green message with a reference number, which prevents duplicate submission when the page is refreshed.

5. What is the difference between GET and POST?

Answer: GET puts data in the URL, can be bookmarked and cached, and is meant for reading data. POST sends data in the request body and is meant for actions that change data. Neither is secure without HTTPS.

6. What is the difference between validation and sanitisation?

Answer: Validation checks whether the data is acceptable and rejects it if not (for example a mobile number with 5 digits). Sanitisation cleans the data (for example removing tags and extra spaces). Both are needed, and output escaping is a third, separate step.

7. What is XSS and how does htmlspecialchars() prevent it?

Answer: Cross-site scripting (XSS) happens when user input containing HTML or JavaScript is shown on a page and executed by the browser. htmlspecialchars() converts characters such as <, >, &, and quotes into HTML entities so that they are displayed as text.

8. Why is flock() used?

Answer: Several requests can arrive at the same time. flock() gives one request exclusive access while it writes, which prevents mixed or lost rows. Readers use a shared lock (LOCK_SH).

9. What is CSRF and how does the token stop it?

Answer: Cross-Site Request Forgery makes a logged-in user's browser send a request to a site without the user knowing. The form contains a random secret token stored in the session; the server accepts a POST only if the submitted token matches the session token (compared with hash_equals()). Another site cannot know the token.

10. What is the Post/Redirect/Get pattern?

Answer: After processing a POST request successfully, the server redirects to a GET page. If the user refreshes, the browser repeats only the harmless GET request and does not resend the form data.

11. Why must the server validate when JavaScript already validates?

Answer: JavaScript runs in the user's browser and can be disabled or bypassed with tools such as curl or the browser developer tools. The server must never trust data from the client.

12. CSV or JSON: which is better?

Answer: CSV is compact, opens in Excel, and is easy to append to. JSON keeps structure and data types and is easy to read from JavaScript. For anything larger or with many simultaneous users, a database is the better choice. This practical writes both: CSV is the master copy and JSON is a second copy.

B. Implementation / Output / Result

Extend the StudentHub project. Copy the folder into the web root of a local server (htdocs in XAMPP, www in WAMP, /var/www/html in LAMP), start Apache, and open http://localhost/StudentHub/register.php. PHP files do not work when opened directly from the file system. PHP 7.4 or newer with the mbstring extension is required (included in XAMPP).

1. Project folder structure

```
StudentHub/
```

```
├── index.html
├── ... (other pages)
├── register.php         (form + POST processing)
├── view.php             (display stored records)
├── css/
│   └── style.css        (previous styles + Practical 7 additions)
├── js/
│   └── script.js        (Practical 4: theme and menu)
├── includes/
│   ├── functions.php    (sanitise, validate, CSRF, file storage)
│   └── layout.php       (shared header and footer)
└── data/
    ├── .htaccess              (blocks direct access)
    ├── registrations.csv      (created automatically)
    └── registrations.json     (created automatically)
```

2. Helper functions (save as includes/functions.php)

```php
<?php
declare(strict_types=1);

/* ============================================================
   functions.php: helpers for Practical 7
   sanitising, validating, CSRF protection and safe file storage
   ============================================================ */

const DATA_DIR  = __DIR__ . '/../data';
const CSV_FILE  = DATA_DIR . '/registrations.csv';
const JSON_FILE = DATA_DIR . '/registrations.json';

const COURSES = [
    'Computer Engineering',
    'Information Technology',
    'Electronics and Communication',
    'Mechanical Engineering',
    'Civil Engineering',
];
const YEARS   = ['1', '2', '3', '4'];
const GENDERS = ['Female', 'Male', 'Other', 'Prefer not to say'];

/* ---------- Output escaping (prevents XSS) ---------- */
function e(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}

/* ---------- Session and CSRF token (advanced extension) ---------- */
function start_secure_session(): void
{
    if (session_status() === PHP_SESSION_NONE) {
        session_set_cookie_params([
            'lifetime' => 0,
            'path'     => '/',
            'httponly' => true,
            'samesite' => 'Lax',
        ]);
        session_start();
    }
}

function csrf_token(): string
{
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}
```

```
function csrf_valid($token): bool
{
    return is_string($token)
        && isset($_SESSION['csrf_token'])
        && hash_equals($_SESSION['csrf_token'], $token);   // timing-safe comparison
}

function csrf_refresh(): void
{
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

/* ---------- Sanitising ---------- */
/* Trim, remove HTML tags and control characters. $multiline keeps line breaks. */
function clean_text($value, bool $multiline = false): string
{
    if (!is_string($value)) {
        return '';                                  // arrays such as name[]=x are rejected
    }
    $value = trim(strip_tags($value));
    $value = (string) preg_replace('/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F]/', '', $value);
    if (!$multiline) {
        $value = (string) preg_replace('/\s+/', ' ', $value);   // collapse tabs/new lines
    }
    return $value;
}

function collect_input(): array
{
    return [
        'fullname' => clean_text($_POST['fullname'] ?? ''),
        'email'    => strtolower(clean_text($_POST['email'] ?? '')),
        'mobile'   => clean_text($_POST['mobile'] ?? ''),
        'course'   => clean_text($_POST['course'] ?? ''),
        'year'     => clean_text($_POST['year'] ?? ''),
        'gender'   => clean_text($_POST['gender'] ?? ''),
        'message'  => clean_text($_POST['message'] ?? '', true),
    ];
}

/* ---------- Validation: returns an array [field => message] ---------- */
function validate_registration(array $in): array
{
    $errors = [];

    if ($in['fullname'] === '') {
        $errors['fullname'] = 'Full name is required.';
    } elseif (!preg_match('/^[A-Za-z][A-Za-z .\'-]{1,49}$/', $in['fullname'])) {
        $errors['fullname'] = 'Name must be 2 to 50 characters: letters, spaces, dots,
apostrophes or hyphens.';
    }

    if ($in['email'] === '') {
        $errors['email'] = 'Email address is required.';
    } elseif (strlen($in['email']) > 254
        || !filter_var($in['email'], FILTER_VALIDATE_EMAIL)
        || !preg_match('/^[^@\s]+@[^@\s]+\.[A-Za-z]{2,}$/', $in['email'])) {
        $errors['email'] = 'Enter a valid email address, for example name@example.com.';
    }

    if ($in['mobile'] === '') {
        $errors['mobile'] = 'Mobile number is required.';
    } elseif (!preg_match('/^[6-9][0-9]{9}$/', $in['mobile'])) {
        $errors['mobile'] = 'Mobile number must be 10 digits and start with 6, 7, 8 or 9.';
    }

    /* Whitelist checks: the value must be one of the allowed options */
    if (!in_array($in['course'], COURSES, true)) {
```

```
        $errors['course'] = 'Select a valid course.';
    }
    if (!in_array($in['year'], YEARS, true)) {
        $errors['year'] = 'Select a valid year.';
    }
    if (!in_array($in['gender'], GENDERS, true)) {
        $errors['gender'] = 'Select a valid option for gender.';
    }

    if (mb_strlen($in['message']) > 300) {
        $errors['message'] = 'Message must not be longer than 300 characters.';
    }

    return $errors;
}

/* ---------- Safe file handling ---------- */
function ensure_data_dir(): bool
{
    return is_dir(DATA_DIR) || mkdir(DATA_DIR, 0750, true);
}

/* Stops spreadsheet formula injection: a cell starting with = + - @ is prefixed with ' */
function csv_safe($value): string
{
    $value = (string) $value;
    return preg_match('/^[=+\-@\t\r]/', $value) ? "'" . $value : $value;
}

function save_csv(array $record): bool
{
    if (!ensure_data_dir()) {
        return false;
    }
    $fp = fopen(CSV_FILE, 'ab');                    // append, create if missing
    if ($fp === false) {
        return false;
    }
    $ok = false;
    if (flock($fp, LOCK_EX)) {                      // exclusive lock: one writer at a time
        $stat = fstat($fp);
        if ($stat !== false && $stat['size'] === 0) {
            fputcsv($fp, array_keys($record), ',', '"', '\\');   // header row for a new file
        }
        $ok = fputcsv($fp, array_map('csv_safe', array_values($record)), ',', '"', '\\') !==
false;
        fflush($fp);
        flock($fp, LOCK_UN);
    }
    fclose($fp);
    return $ok;
}

function save_json(array $record): bool
{
    if (!ensure_data_dir()) {
        return false;
    }
    $fp = fopen(JSON_FILE, 'c+');                   // read/write, create if missing, no
truncate
    if ($fp === false) {
        return false;
    }
    $ok = false;
    if (flock($fp, LOCK_EX)) {
        $raw  = stream_get_contents($fp);
        $rows = [];
        if ($raw !== false && trim($raw) !== '') {
            $decoded = json_decode($raw, true);
```

```
            if (is_array($decoded)) {
                $rows = $decoded;
            } else {
                flock($fp, LOCK_UN);                // existing file is corrupt: do not
overwrite it
                fclose($fp);
                return false;
            }
        }
        $rows[] = $record;
        $json = json_encode($rows, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE |
JSON_UNESCAPED_SLASHES);
        if ($json !== false) {
            ftruncate($fp, 0);
            rewind($fp);
            $ok = fwrite($fp, $json) !== false;
            fflush($fp);
        }
        flock($fp, LOCK_UN);
    }
    fclose($fp);
    return $ok;
}

/* Read every stored record from the CSV file as an array of associative arrays */
function read_csv_records(): array
{
    if (!is_readable(CSV_FILE)) {
        return [];
    }
    $fp = fopen(CSV_FILE, 'rb');
    if ($fp === false) {
        return [];
    }
    $rows   = [];
    $header = null;
    if (flock($fp, LOCK_SH)) {                      // shared lock: readers do not block each
other
        while (($row = fgetcsv($fp, 0, ',', '"', '\\')) !== false) {
            if ($row === [null]) {
                continue;                           // blank line
            }
            if ($header === null) {
                $header = $row;
                continue;
            }
            if (count($row) === count($header)) {
                $rows[] = array_combine($header, $row);
            }
        }
        flock($fp, LOCK_UN);
    }
    fclose($fp);
    return $rows;
}

function email_exists(string $email): bool
{
    foreach (read_csv_records() as $row) {
        if (strcasecmp($row['email'] ?? '', $email) === 0) {
            return true;
        }
    }
    return false;
}
```

3. Shared layout (save as includes/layout.php)

```
<?php
declare(strict_types=1);
```

```
/* layout.php: shared header and footer so every PHP page looks the same */

function render_header(string $title, string $current): void
{
    $links = [
        'index.html'    => 'Home',
        'register.php'  => 'Register',
        'view.php'      => 'Records',
        'events.html'   => 'Events',
        'contact.html'  => 'Contact',
    ];
    ?>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><?= e($title) ?> | StudentHub</title>
  <link rel="stylesheet" href="css/style.css">
  <script src="js/script.js" defer></script>
</head>
<body>
  <a class="skip-link" href="#main-content">Skip to main content</a>
  <header class="site-header">
    <div class="container header-row">
      <p class="brand"><strong>StudentHub</strong></p>
      <div class="header-controls">
        <button class="icon-btn" id="theme-toggle" type="button" aria-label="Dark theme" aria-
pressed="false">Dark</button>
        <button class="icon-btn menu-toggle" id="menu-toggle" type="button" aria-label="Toggle
navigation menu" aria-expanded="false" aria-controls="main-nav">&#9776;</button>
      </div>
      <nav class="main-nav" id="main-nav" aria-label="Main navigation">
        <ul class="nav-list">
<?php foreach ($links as $href => $label): ?>
          <li><a href="<?= e($href) ?>"<?= $href === $current ? ' aria-current="page"' : ''
?>><?= e($label) ?></a></li>
<?php endforeach; ?>
        </ul>
      </nav>
    </div>
  </header>
  <main id="main-content" class="container">
<?php
}

function render_footer(): void
{
    ?>
  </main>
  <footer class="site-footer">
    <div class="container">
      <p>&copy; 2026 StudentHub | PHP Form Processing with Server-Side Validation</p>
    </div>
  </footer>
</body>
</html>
<?php
}
```

4. Form and processing script (save as register.php)

```
<?php
declare(strict_types=1);

require __DIR__ . '/includes/functions.php';
require __DIR__ . '/includes/layout.php';
```

```
start_secure_session();

$errors    = [];
$formError = '';
$old       = ['fullname' => '', 'email' => '', 'mobile' => '', 'course' => '', 'year' => '',
'gender' => '', 'message' => ''];

/* Flash message left by the previous request (Post/Redirect/Get pattern) */
$flash = $_SESSION['flash'] ?? null;
unset($_SESSION['flash']);

$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';

if ($method === 'POST') {
    if (!csrf_valid($_POST['csrf_token'] ?? null)) {
        http_response_code(403);
        $formError = 'Security check failed: the form token is missing or has expired. Please
reload the page and try again.';
    } else {
        $old    = collect_input();                      // 1. sanitise
        $errors = validate_registration($old);          // 2. validate

        if (!$errors && email_exists($old['email'])) {   // 3. business rule: unique email
            $errors['email'] = 'This email address is already registered.';
        }

        if ($errors) {
            http_response_code(422);
        } else {
            $record = [
                'id'           => date('YmdHis') . bin2hex(random_bytes(2)),
                'submitted_at' => date('Y-m-d H:i:s'),
                'fullname'     => $old['fullname'],
                'email'        => $old['email'],
                'mobile'       => $old['mobile'],
                'course'       => $old['course'],
                'year'         => $old['year'],
                'gender'       => $old['gender'],
                'message'      => $old['message'],
            ];

            if (save_csv($record)) {                    // CSV is the master copy
                if (!save_json($record)) {
                    error_log('StudentHub: JSON copy could not be written for ' .
$record['id']);
                }
                csrf_refresh();                         // a token is used only once
                $_SESSION['flash'] = [
                    'type' => 'success',
                    'text' => 'Registration saved. Thank you, ' . $record['fullname'] . '! Your
reference number is ' . $record['id'] . '.',
                ];
                header('Location: register.php');       // redirect so a refresh cannot
resubmit the form
                exit;
            }

            http_response_code(500);
            error_log('StudentHub: CSV file could not be written.');
            $formError = 'Sorry, your registration could not be saved. Please try again
later.';
        }
    }
} elseif ($method !== 'GET') {
    http_response_code(405);
    header('Allow: GET, POST');
    exit('Method not allowed.');
}
```

```
render_header('Registration', 'register.php');
?>
    <section class="panel form-panel" aria-labelledby="reg-title">
      <h1 id="reg-title">Student Registration</h1>
      <p class="muted">Fields marked * are required. All data is checked again on the
server.</p>

<?php if ($flash): ?>
      <div class="alert <?= e($flash['type']) ?>" role="status"><?= e($flash['text']) ?></div>
<?php endif; ?>

<?php if ($formError !== ''): ?>
      <div class="alert error" role="alert"><?= e($formError) ?></div>
<?php endif; ?>

<?php if ($errors): ?>
      <div class="error-summary" role="alert">
        <h2>Please correct <?= count($errors) ?> error<?= count($errors) > 1 ? 's' : '' ?></h2>
        <ul>
<?php foreach ($errors as $field => $message): ?>
          <li><a href="#<?= e($field === 'gender' ? 'gender-female' : $field) ?>"><?=
e($message) ?></a></li>
<?php endforeach; ?>
        </ul>
      </div>
<?php endif; ?>

      <!-- novalidate lets the server-side checks be tested without browser pop-ups -->
      <form action="register.php" method="post" novalidate>
        <input type="hidden" name="csrf_token" value="<?= e(csrf_token()) ?>">

        <div class="field<?= isset($errors['fullname']) ? ' has-error' : '' ?>">
          <label for="fullname">Full name *</label>
          <input type="text" id="fullname" name="fullname" required maxlength="50"
autocomplete="name"
                 value="<?= e($old['fullname']) ?>" aria-describedby="fullname-error"
                 aria-invalid="<?= isset($errors['fullname']) ? 'true' : 'false' ?>">
          <p class="error-msg" id="fullname-error"><?= e($errors['fullname'] ?? '') ?></p>
        </div>

        <div class="field<?= isset($errors['email']) ? ' has-error' : '' ?>">
          <label for="email">Email address *</label>
          <input type="email" id="email" name="email" required maxlength="254"
autocomplete="email"
                 value="<?= e($old['email']) ?>" aria-describedby="email-error"
                 aria-invalid="<?= isset($errors['email']) ? 'true' : 'false' ?>">
          <p class="error-msg" id="email-error"><?= e($errors['email'] ?? '') ?></p>
        </div>

        <div class="field<?= isset($errors['mobile']) ? ' has-error' : '' ?>">
          <label for="mobile">Mobile number *</label>
          <input type="tel" id="mobile" name="mobile" required maxlength="10"
inputmode="numeric" autocomplete="tel-national"
                 value="<?= e($old['mobile']) ?>" aria-describedby="mobile-error"
                 aria-invalid="<?= isset($errors['mobile']) ? 'true' : 'false' ?>">
          <p class="error-msg" id="mobile-error"><?= e($errors['mobile'] ?? '') ?></p>
        </div>

        <div class="field-row">
          <div class="field<?= isset($errors['course']) ? ' has-error' : '' ?>">
            <label for="course">Course *</label>
            <select id="course" name="course" required aria-describedby="course-error"
                    aria-invalid="<?= isset($errors['course']) ? 'true' : 'false' ?>">
              <option value="">Select a course</option>
<?php foreach (COURSES as $course): ?>
```

```
            <option value="<?= e($course) ?>"<?= $old['course'] === $course ? ' selected' :
'' ?>><?= e($course) ?></option>
<?php endforeach; ?>
          </select>
          <p class="error-msg" id="course-error"><?= e($errors['course'] ?? '') ?></p>
        </div>

        <div class="field<?= isset($errors['year']) ? ' has-error' : '' ?>">
          <label for="year">Year *</label>
          <select id="year" name="year" required aria-describedby="year-error"
                  aria-invalid="<?= isset($errors['year']) ? 'true' : 'false' ?>">
            <option value="">Select a year</option>
<?php foreach (YEARS as $year): ?>
            <option value="<?= e($year) ?>"<?= $old['year'] === $year ? ' selected' : ''
?>>Year <?= e($year) ?></option>
<?php endforeach; ?>
          </select>
          <p class="error-msg" id="year-error"><?= e($errors['year'] ?? '') ?></p>
        </div>
      </div>

      <fieldset class="field choice-group<?= isset($errors['gender']) ? ' has-error' : '' ?>"
aria-describedby="gender-error">
        <legend>Gender *</legend>
<?php foreach (GENDERS as $i => $gender): ?>
        <label class="choice"><input type="radio" id="<?= $i === 0 ? 'gender-female' :
'gender-' . $i ?>" name="gender" value="<?= e($gender) ?>"<?= $old['gender'] === $gender ? '
checked' : '' ?>> <?= e($gender) ?></label>
<?php endforeach; ?>
        <p class="error-msg" id="gender-error"><?= e($errors['gender'] ?? '') ?></p>
      </fieldset>

      <div class="field<?= isset($errors['message']) ? ' has-error' : '' ?>">
        <label for="message">Message (optional, up to 300 characters)</label>
        <textarea id="message" name="message" rows="4" maxlength="300" aria-
describedby="message-error"
                  aria-invalid="<?= isset($errors['message']) ? 'true' : 'false' ?>"><?=
e($old['message']) ?></textarea>
        <p class="error-msg" id="message-error"><?= e($errors['message'] ?? '') ?></p>
      </div>

      <div class="form-actions">
        <button class="button" type="submit">Submit registration</button>
        <a class="button secondary" href="view.php">View stored records</a>
      </div>
    </form>
  </section>
<?php render_footer(); ?>
```

5. Stored records page (save as view.php)

```
<?php
declare(strict_types=1);

require __DIR__ . '/includes/functions.php';
require __DIR__ . '/includes/layout.php';

/* Intermediate extension: show the stored CSV records on a web page.
   In a real system this page must be protected by a login. */
$records = array_reverse(read_csv_records());       // newest first

render_header('Stored Records', 'view.php');
?>
  <h1>Stored Registrations</h1>
  <p class="muted" role="status"><?= count($records) ?> record<?= count($records) === 1 ? ''
: 's' ?> found in data/registrations.csv</p>
```

```
<?php if (!$records): ?>
    <p class="empty">No registrations yet. <a href="register.php">Submit the first one.</a></p>
<?php else: ?>
    <div class="table-wrap">
      <table class="data-table">
        <caption class="visually-hidden">Registrations stored in the CSV file, newest
first</caption>
        <thead>
          <tr>
            <th scope="col">Submitted</th>
            <th scope="col">Name</th>
            <th scope="col">Email</th>
            <th scope="col">Mobile</th>
            <th scope="col">Course</th>
            <th scope="col">Year</th>
            <th scope="col">Gender</th>
            <th scope="col">Message</th>
          </tr>
        </thead>
        <tbody>
<?php foreach ($records as $row): ?>
          <tr>
            <td><?= e($row['submitted_at'] ?? '') ?></td>
            <td><?= e($row['fullname'] ?? '') ?></td>
            <td><?= e($row['email'] ?? '') ?></td>
            <td><?= e($row['mobile'] ?? '') ?></td>
            <td><?= e($row['course'] ?? '') ?></td>
            <td><?= e($row['year'] ?? '') ?></td>
            <td><?= e($row['gender'] ?? '') ?></td>
            <td><?= e($row['message'] ?? '') ?></td>
          </tr>
<?php endforeach; ?>
        </tbody>
      </table>
    </div>
<?php endif; ?>
<?php render_footer(); ?>
```

6. Data folder protection (save as data/.htaccess)

```
# Block direct browser access to stored records (Apache 2.4)
Require all denied
```

7. Additional CSS (append to css/style.css)

```
/* ===== Practical 7 additions: append to css/style.css ===== */
/* (form, .alert, .error-summary, .visually-hidden and colour tokens come from Practical 5;
   .empty comes from Practical 6) */

textarea { width: 100%; padding: .6rem .7rem; font: inherit; color: var(--text); background:
var(--surface); border: 2px solid var(--field-border); border-radius: .4rem; resize: vertical;
}
textarea:focus-visible { outline: 3px solid #f5b700; outline-offset: 1px; }
.has-error textarea { border-color: var(--danger); background: var(--danger-bg); }
.empty { padding: 1.5rem; text-align: center; background: var(--surface); border: 1px dashed
var(--border); border-radius: .75rem; }

/* Records table */
.table-wrap { overflow-x: auto; border: 1px solid var(--border); border-radius: .5rem; }
.data-table { width: 100%; border-collapse: collapse; background: var(--surface); }
.data-table th, .data-table td { padding: .6rem .75rem; text-align: left; border-bottom: 1px
solid var(--border); vertical-align: top; }
.data-table thead th { background: var(--primary); color: #fff; white-space: nowrap; }
.data-table tbody tr:nth-child(even) { background: var(--page-bg); }
```

10. Sample of the generated files

After a few submissions the data folder contains files like the following. (Example content; your references and times will differ.)

data/registrations.csv

```
id,submitted_at,fullname,email,mobile,course,year,gender,message
20261105103412a3f1,"2026-11-05 10:34:12","Riya
Patel",riya.patel@example.com,9876543210,"Computer Engineering",2,Female,"Please share the
workshop timetable."
20261105104009c8d2,"2026-11-05 10:40:09","Aarav
Shah",aarav.shah@example.com,9123456780,"Information Technology",3,Male,
20261105110527e41b,"2026-11-05 11:05:27","O'Neil
Dev",dev.oneil@example.com,8899776655,"Mechanical Engineering",1,Other,"'=SUM(A1:A5)"
```

data/registrations.json


```
[
    {
        "id": "20261105103412a3f1",
        "submitted_at": "2026-11-05 10:34:12",
        "fullname": "Riya Patel",
        "email": "riya.patel@example.com",
        "mobile": "9876543210",
        "course": "Computer Engineering",
        "year": "2",
        "gender": "Female",
        "message": "Please share the workshop timetable."
    },
    {
        "id": "20261105104009c8d2",
        "submitted_at": "2026-11-05 10:40:09",
        "fullname": "Aarav Shah",
        "email": "aarav.shah@example.com",
        "mobile": "9123456780",
        "course": "Information Technology",
        "year": "3",
        "gender": "Male",
        "message": ""
    }
]
```


C. Evidence of Work (Screenshots)

Replace each frame below with your own screenshot. Also include the PHP source files in VS Code and the generated CSV and JSON files.

D. Server-Side Processing Checklist

☐ The form uses method="post" and the script checks REQUEST_METHOD.

☐ All inputs are read from $_POST, trimmed, and cleaned of tags and control characters.

☐ Name, email, and mobile are validated with regular expressions or filter_var().

☐ Course, year, and gender are checked against whitelists.

☐ Validation errors are shown beside the fields and the entered values are kept.

☐ Output is escaped with htmlspecialchars() everywhere.

☐ Files are written with fopen(), flock(), and fputcsv()/json_encode(), and failures are handled.

☐ File names are fixed in the code and not taken from user input.

☐ The data folder is protected from direct download.

☐ A success message is shown after Post/Redirect/Get.

☐ Duplicate emails are rejected.

☐ The CSRF token is generated, verified, and refreshed.

☐ Appropriate HTTP status codes are returned.

☐ The CSV and JSON files are created and contain the correct values.

☐ Stored records are displayed on a webpage with escaped values.

E. Challenges Faced

1. Deciding which checks belong to sanitisation, validation, and output escaping.
2. Preventing two simultaneous submissions from corrupting the file (solved with flock).
3. Keeping the user's entered values after an error without opening an XSS hole (solved with e()).
4. Avoiding duplicate records on page refresh (solved with Post/Redirect/Get).
5. Making sure PHP files are opened through localhost and not directly from the disk.
6. Protecting the data files from being downloaded.

F. Learning Outcomes

1. Processed form data on the server with the POST method and PHP superglobals.
2. Applied server-side validation and sanitisation, and understood why client-side checks are not enough.
3. Stored data safely in CSV and JSON files using file locking and error handling.
4. Displayed clear success and error messages with correct HTTP status codes.
5. Protected the form against XSS, CSRF, CSV injection, and direct file access.

G. Viva Questions

Which superglobal holds data sent by a POST form?

Answer: $_POST.

What does strip_tags() do?

Answer: It removes HTML and PHP tags from a string.

What does htmlspecialchars() do?

Answer: It converts special characters to HTML entities so they are shown as text and not executed.

Why use in_array() with the third argument true?

Answer: It makes the comparison strict (type and value), which is safer for whitelist checks.

What is the difference between fopen modes ‘a’ and ‘w’?

Answer: ‘a’ appends to the end of the file; ‘w’ empties the file first.

What does LOCK_EX do?

Answer: It gives the process exclusive access to the file, so other writers wait.

Why is header('Location: ...') followed by exit?

Answer: The script must stop after redirecting so that no further code runs.

What does fputcsv() do that implode() does not?

Answer: It correctly quotes values that contain commas, quotes, or new lines.

What is CSV injection?

Answer: A cell that starts with =, +, -, or @ may be run as a formula by a spreadsheet; the code prefixes such cells with an apostrophe.

Why is hash_equals() used for tokens?

Answer: It compares in constant time, which prevents timing attacks.

What does HTTP status 422 mean?

Answer: The request was understood but the data is not valid (unprocessable).

Why store password hashes and never plain passwords?

Answer: If the file leaks, hashes made with password_hash() cannot be reversed easily. This form does not store passwords at all.

H. Result

A PHP script receives the registration/contact form with POST, sanitises and validates every input on the server, protects the form with a CSRF token, stores each valid record safely in CSV and JSON files, and shows clear success and error messages. The stored records are displayed on a webpage. The test table, the generated files, and the screenshots demonstrate the final result.

Faculty Remarks

(Keep this section blank for faculty.)